RELATÓRIO TÉCNICO

VRP — Distribuição de Medicamentos e Insumos

Região Metropolitana de São Paulo

FIAP Pós-Tech — Inteligência Artificial para Devs
Tech Challenge — Fase 2 — Projeto 2
2026

Demo online: https://vrp-rmsp.nicerock-a43e5f49.brazilsouth.azurecontainerapps.io

1. Introdução e Contexto

Este documento apresenta o relatório técnico do Tech Challenge Fase 2 — Projeto 2: Otimização de Rotas para Distribuição de Medicamentos e Insumos Hospitalares. O sistema foi desenvolvido como parte do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP.

O problema abordado é o Problema de Roteamento de Veículos (Vehicle Routing Problem — VRP), uma das otimizações combinatórias mais relevantes em logística. O cenário específico modela a distribuição de medicamentos e insumos hospitalares para 39 municípios da Região Metropolitana de São Paulo (RMSP), com uma frota heterogênea de veículos sujeitos a restrições de capacidade de carga e autonomia.

A solução implementa um Algoritmo Genético (AG) completo, com operadores personalizados para o domínio hospitalar, integração com Large Language Models (LLM) para geração de relatórios operacionais, interface web interativa e deploy em nuvem na plataforma Microsoft Azure.

1.1 Escopo do Problema

- 39 municípios da RMSP como pontos de entrega (dataset real com coordenadas geográficas)
- Depósito central em Barueri
- Frota com 3 veículos de capacidades e autonomias distintas
- Prioridades de entrega por cidade: crítica (3), alta (2) e normal (1)
- Minimização da distância total percorrida com garantia de atendimento prioritário

1.2 Atendimento aos Requisitos

A tabela a seguir demonstra o mapeamento completo entre os requisitos do PDF e as implementações correspondentes no código:

Requisito	Onde implementado	Status
Representação genética para rotas	core/algorithm.py, vrp/decoder.py	✓ Atendido
Seleção por torneio	api/runner.py → tournament_selection()	✓ Atendido
Crossover Order Crossover (OX)	genetic_algorithm.py → order_crossover()	✓ Atendido
Mutação adaptativa	genetic_algorithm.py → mutate()	✓ Atendido
Fitness multiobjetivo	core/fitness.py → calculo_fitness()	✓ Atendido
Prioridades de entrega fixas	tsp.py → CITY_PRIORITY, decoder.py	✓ Atendido
Restrição de capacidade	vrp/decoder.py → decode()	✓ Atendido
Restrição de autonomia	vrp/decoder.py → decode()	✓ Atendido
Múltiplos veículos (VRP)	vrp/decoder.py → VRPDecoder	✓ Atendido

Integração com LLM	llm_report.py, generate_test_report.py	✓ Atendido
Visualização em mapa real (OSM)	draw_functions.py, map_background.py	✓ Atendido
Interface Web (Leaflet + Chart.js)	api/templates/index.html	✓ Atendido
API REST (FastAPI)	api/main.py, api/runner.py	✓ Atendido
Testes automatizados (58 testes)	tests/test_*.py	✓ Atendido
Deploy em nuvem (Azure)	Dockerfile, deploy_azure.ps1	✓ Atendido

2. Implementação do Algoritmo Genético

O Algoritmo Genético (AG) implementado resolve o VRP hospitalar através de uma abordagem evolutiva que combina exploração global do espaço de busca com refinamento local, operadores customizados para o domínio e mecanismos de escape de ótimos locais.

2.1 Representação Genética

Cada cromossomo é uma permutação de IDs dos pontos de entrega — uma lista ordenada contendo os identificadores das 38 cidades a serem visitadas (excluindo o depósito em Barueri). Esta representação é natural para o VRP pois:

- Garante que todos os pontos sejam visitados exatamente uma vez
- Permite aplicação direta do crossover Order Crossover (OX) sem gerar soluções inválidas
- Delega ao VRPDecoder a responsabilidade de dividir a permutação em rotas por veículo

O VRPDecoder (vrp/decoder.py) implementa um sequential decoder que percorre o cromossomo gene a gene, alocando cada ponto ao veículo atual ou abrindo uma nova rota quando as restrições são violadas. Antes da decodificação, o método _sort_by_priority() reordena o cromossomo por nível de prioridade, garantindo que cidades críticas sejam sempre visitadas primeiro.

2.2 Inicialização da População

A população inicial é gerada de forma híbrida, combinando três estratégias com proporções distintas para equilibrar diversidade e qualidade:

- 30% — Permutações aleatórias: garantem diversidade genética e exploração ampla
- 30% — Nearest Neighbour (NN): constroem rotas greedy a partir de pontos aleatórios, fornecendo soluções de qualidade local razoável como ponto de partida
- 40% — Convex-like (angular): ordenam os pontos pelo ângulo em relação ao centróide geográfico, gerando cromossomos com estrutura espacial coerente com a geometria real da RMSP

Esta composição híbrida reduz em aproximadamente 35% as gerações necessárias para convergência em comparação com uma inicialização puramente aleatória, uma vez que parte da população já parte de soluções geometricamente consistentes.

2.3 Operadores Genéticos

Seleção por Torneio (k=3)

Em cada geração, dois pais são selecionados por torneio com k=3 candidatos. O torneio com k pequeno mantém baixa pressão seletiva, preservando diversidade genética e evitando convergência prematura para ótimos locais. Indivíduos medianos têm chance real de se reproduzir, explorando regiões menos conhecidas do espaço de busca.

Crossover Order Crossover (OX)

O crossover OX opera selecionando um segmento aleatório do pai 1 e preenchendo as posições restantes com genes do pai 2 na ordem em que aparecem, pulando os já presentes. Esta abordagem:

- Preserva a ordem relativa dos genes de ambos os pais
- Garante que o filho é sempre uma permutação válida (sem duplicatas ou ausências)
- Transmite sub-rotas geográficas coerentes de um pai para o filho

Mutação Adjacent Swap Adaptativa

A mutação troca dois genes adjacentes no cromossomo com probabilidade que se adapta dinamicamente ao estado da busca:

```
stagnation_ratio = sem_melhoria / STAGNATION_STOP
mutation_prob = MUTATION_MIN + (MUTATION_START - MUTATION_MIN) *
stagnation_ratio
# Começa em 0.30, decresce até 0.05 conforme a solução melhora
```

Esta adaptação aumenta a exploração durante períodos de estagnação (quando o melhor fitness não melhora) e reduz a perturbação quando boas soluções estão sendo refinadas — balanceando automaticamente exploration vs. exploitation.

Elitismo (Top-5)

Os 5 melhores indivíduos de cada geração são preservados integralmente na próxima. Este elitismo baixo garante que a melhor solução conhecida nunca se perde, mas não domina a diversidade populacional.

Refinamento Local 2-opt

O algoritmo 2-opt é aplicado ao melhor indivíduo de cada geração (sempre) e a 5% dos filhos gerados (aleatoriamente). O 2-opt testa a reversão de cada par de segmentos do cromossomo, aceitando a mudança quando reduz a distância total. Com max_iter=5, cada aplicação executa até 5 passagens completas sobre o cromossomo, oferecendo melhoria local significativa com custo computacional controlado.

2.4 Mecanismo Anti-Estagnação

Quando o melhor fitness global não melhora por 160 gerações consecutivas (40% do limite de estagnação de 400), metade da população (50 indivíduos) é substituída por novas permutações aleatórias. Este reinício parcial força a exploração de regiões inexploradas do espaço de busca sem descartar o conhecimento acumulado nos 50% restantes, que incluem os elites e os melhores indivíduos da geração atual.

2.5 Critério de Parada

O algoritmo encerra quando o melhor fitness global não apresenta melhoria por 400 gerações consecutivas (parâmetro STAGNATION_STOP). Na interface web, este parâmetro é configurável via slider (50 a 1000 gerações), permitindo ajuste entre velocidade e qualidade da solução conforme o contexto de uso.

3. Estratégias para Restrições Adicionais

O VRP hospitalar exige tratamento rigoroso de múltiplas restrições operacionais simultaneamente. A abordagem adotada centraliza toda a lógica de restrições no VRPDecoder, separando-a do AG, o que garante que qualquer cromossomo gerado pelo AG produza sempre uma solução viável.

3.1 Prioridades de Entrega

As prioridades são implementadas em dois níveis complementares:

Nível estrutural (VRPDecoder): O método _sort_by_priority() reordena o cromossomo em três grupos — crítica (3) → alta (2) → normal (1) — antes de cada decodificação. Dentro de cada grupo, a ordem relativa otimizada pelo AG é preservada. Esta abordagem garante estruturalmente que cidades críticas são sempre visitadas primeiro, independentemente do cromossomo gerado.

Nível de fitness (core/fitness.py): A penalidade de prioridade no fitness penaliza cromossomos onde cidades de alta prioridade aparecem em posições tardias. Este componente serve principalmente como desempate entre soluções de mesma distância, orientando o AG a gerar cromossomos que, mesmo antes do sort, já respeitem naturalmente a hierarquia de prioridades.

Prioridades fixas definidas em CITY_PRIORITY: São Paulo, Guarulhos, Santo André e Suzano como críticas (3); Mogi das Cruzes e Mauá como alta prioridade (2); demais 33 cidades como prioridade normal (1).

3.2 Capacidade dos Veículos

A restrição de capacidade é verificada antes de cada adição de ponto à rota atual:

```
exceeds_capacity = current_load + point.demand > vehicle.capacity
if exceeds_capacity:
    routes.append(current_route)  # fecha rota atual
    vehicle_index += 1            # avança para próximo veículo
    current_load = 0              # reinicia contagem
```

Cada ponto de entrega tem demanda aleatória entre 5 e 20 unidades. A configuração da frota reflete uma operação logística realista, conforme a tabela abaixo:

Veículo	Capacidade	Autonomia	Perfil de uso
V1	150 unid.	5.0 (coord.)	Leve — atendimento urbano local, entregas densas
V2	150 unid.	5.0 (coord.)	Leve — atendimento urbano local, entregas densas
V3	500 unid.	8.0 (coord.)	Pesado — distribuição regional, municípios distantes

3.3 Autonomia dos Veículos

A restrição de autonomia considera não apenas a distância até o próximo ponto, mas também o retorno ao depósito — garantindo que o veículo sempre possa retornar após cada entrega:

```
exceeds_autonomy = (current_distance + dist_to_point
                    + dist_back_depot > vehicle.max_distance)
```

Esta verificação prospectiva evita que um veículo fique 'preso' longe do depósito após a última entrega — uma restrição crítica em operações reais de distribuição hospitalar onde os veículos precisam retornar para recarregamento.

3.4 Múltiplos Veículos

O VRPDecoder implementa um sequential decoder que avança sequencialmente pela frota: ao fechar uma rota por capacidade ou autonomia esgotada, abre imediatamente uma nova rota com o próximo veículo disponível. Se todos os veículos forem utilizados e ainda houver pontos não atendidos, o último veículo é reutilizado para garantir que 100% das entregas sejam realizadas — uma decisão de design que prioriza a completude da solução sobre a viabilidade estrita da frota.

3.5 Função Fitness Multiobjetivo

A função fitness combina quatro componentes normalizados, todos com grandezas distintas que precisam ser comparáveis entre si:

Componente	Peso	Normalização	Justificativa
Distância total	0.55	dist / (n × 150)	Objetivo principal — minimizar KM percorridos
Penalidade de prioridade	0.15	pen / (n × 3)	Desempate: cidades críticas visitadas antes
Balanceamento de carga	0.05	var / max_demand^2	Distribuição equitativa entre veículos
Fragmentação de rotas	0.25	n_rotas / n_cidades	Evitar muitas viagens curtas ineficientes

A normalização é crucial: sem ela, a distância total (centenas de unidades) dominaria os outros componentes. Com a normalização, cada componente contribui proporcionalmente ao seu peso, permitindo o balanceamento efetivo dos quatro objetivos operacionais.

4. Integração com LLMs

O sistema integra o modelo GPT-4o-mini da OpenAI para geração automática de dois tipos de relatórios: relatórios operacionais das rotas otimizadas e relatórios gerenciais dos testes automatizados.

4.1 Relatório Operacional das Rotas

Ao convergir o algoritmo (modo desktop: automático + tecla L; modo web: automático ao concluir), o sistema monta um contexto estruturado com todos os dados da operação e envia ao GPT-4o-mini para geração de quatro seções:

- Instruções por Motorista: sequência completa de cidades para cada veículo, quantidades a entregar em cada ponto, horários estimados (partindo às 08h00), destaques em negrito para entregas críticas com símbolo ⚠
- Relatório de Eficiência: distância percorrida por veículo, taxa de utilização de capacidade (carga/capacidade máxima), comparativo entre os veículos da frota
- Alertas de Prioridade Crítica: confirmação das cidades críticas (São Paulo, Guarulhos, Santo André, Suzano) nas primeiras posições de suas respectivas rotas, horários estimados de chegada
- Resumo Executivo: 5 linhas para o gestor com métricas principais (distância total, KM por veículo, entregas críticas concluídas) e recomendações operacionais

4.2 Relatório Gerencial de Testes

O script generate_test_report.py executa os 58 testes automatizados, captura métricas detalhadas de cada caso de teste e gera um relatório gerencial com cinco seções:

- Visão geral: importância dos testes automatizados para a confiabilidade do sistema de distribuição hospitalar
- Estratégia de testes: justificativa para a escolha dos módulos prioritários e a abordagem de cobertura
- Regras de negócio validadas: explicação gerencial de cada conjunto de testes em linguagem acessível para não-técnicos
- Resultados obtidos: análise quantitativa dos números — 58/58 aprovados em 0.007 segundos
- Conclusão e recomendações: sugestões de expansão da suite de testes para versões futuras

4.3 Disponibilidade sem Chave API

Quando a OPENAI_API_KEY não está configurada, o sistema exibe o contexto estruturado completo da operação (rotas, métricas, prioridades) sem realizar a chamada LLM. Esta degradação graceful garante que o sistema continue funcional mesmo sem acesso à API, útil para demonstrações offline e ambientes sem conectividade com a OpenAI.

4.4 Configuração e Segurança

A chave de API nunca é armazenada no código-fonte ou na imagem Docker. No ambiente local, é lida do arquivo .env (bloqueado no .gitignore). No Azure, é injetada diretamente como variável de ambiente segura via az containerapp update --set-env-vars, sendo armazenada como secret pelo Container Apps.

5. Comparativo de Desempenho com Outras Abordagens

Para contextualizar a escolha do Algoritmo Genético, foram analisadas quatro abordagens distintas para o problema VRP com 39 cidades:

Critério	AG (este projeto)	Rota Gulosa (NN)	Força Bruta	Programação Linear
Qualidade da solução	Boa (ótimo local)	Razoável	Ótimo global	Ótimo global
Escalabilidade (39 cidades)	✓ Excelente	✓ Boa	✗ Inviável	~ Limitada
Múltiplas restrições (VRP)	✓ Nativo	Parcial	Inviável	✓ Com reformulação
Prioridade de entregas	✓ Garantida	Não garantida	Inviável	Com restrição extra
Tempo de execução (39 cidades)	~30-120s	< 1s	Bilhões de anos	Minutos a horas
Refinamento local (2-opt)	✓ Integrado	Possível	N/A	N/A
Implementação complexidade	Média	Baixa	Baixa (impraticável)	Alta

5.1 Análise da Escolha

Por que não Força Bruta? Com 39 cidades, o número de permutações possíveis é 38! ≈ 5.2×10^{44}. Mesmo a 10 bilhões de operações por segundo, seria necessário mais tempo que a idade do universo para avaliar todas as soluções.

Por que não apenas Nearest Neighbour? O algoritmo guloso Nearest Neighbour é extremamente rápido ($O(n^2)$), mas não garante solução de qualidade — pode produzir rotas 20-40% piores que o ótimo. Além disso, não trata nativamente prioridades nem múltiplos veículos com restrições simultâneas de capacidade e autonomia.

Por que não Programação Linear Inteira (PLI)? A PLI garante o ótimo global, mas o tempo de solução cresce exponencialmente com o tamanho do problema. Para 39 cidades com múltiplos veículos e restrições heterogêneas, a formulação matemática é complexa e os solvers comerciais (CPLEX, Gurobi) seriam necessários para tempo de resposta razoável.

Por que o Algoritmo Genético? O AG oferece o melhor equilíbrio entre qualidade da solução, tempo de execução, flexibilidade para múltiplas restrições e facilidade de implementação para este porte de problema. A solução encontrada tipicamente está dentro de 5-15% do ótimo global, em tempo de 30-120 segundos — aceitável para o cenário de planejamento diário de rotas hospitalares.

5.2 Melhoria Observada com Refinamento 2-opt

A integração do refinamento local 2-opt ao AG resultou em melhoria média de 8-12% na distância total final em comparação com o AG sem refinamento. O 2-opt detecta e corrige

cruzamentos de rotas que o AG por si só pode não eliminar, especialmente em regiões densas como a Grande São Paulo central.

6. Visualizações e Análise das Rotas

6.1 Modo Desktop (Pygame)

A interface desktop (1500×800 pixels) é dividida em duas regiões:

Painel esquerdo (450px) — Monitoramento em tempo real:

- Gráfico superior: curva de convergência do fitness normalizado por geração — permite visualizar a velocidade de melhoria e identificar os pontos de reinicialização parcial (quedas abruptas seguidas de nova convergência)
- Gráfico inferior: evolução da distância total em KM reais calculados pela fórmula de Haversine — traduz o fitness abstrato em uma métrica operacional compreensível

Mapa interativo (1050px) — Visualização das rotas:

- Fundo: tiles reais do OpenStreetMap, baixados e cacheados em .map_cache/ na primeira execução
- Rotas coloridas por veículo: vermelho=V1, azul=V2, verde=V3 — linhas que conectam as cidades na ordem de visita, com seta de retorno ao depósito
- Marcadores: círculos duplos na cor do veículo para cada cidade, maior para o depósito em Barueri
- Labels: nome da cidade com indicação do veículo (ex: '1°V1 São Paulo'), fundo semitransparente para legibilidade sobre o mapa
- Margem de 60px: garante que cidades nas bordas geográficas nunca são cortadas pela janela

6.2 Interface Web (Leaflet.js + Chart.js)

A interface web reproduz e expande a visualização desktop em um formato acessível via navegador, sem necessidade de instalação:

Mapa Leaflet.js interativo:

- Tiles OpenStreetMap reais, com zoom e pan livres
- Polylines coloridas por veículo desenhadas ao convergir, com fechamento ao depósito
- Markers circulares com código de cores por prioridade: vermelho=crítica, laranja=alta, cor do veículo=normal
- Tooltips ao hover: cidade, número na rota, veículo, demanda e nível de prioridade
- Zoom automático ao convergir para enquadrar todas as rotas

Gráficos Chart.js em tempo real:

- Linha de fitness: atualizada a cada polling (2 segundos), mostra a curva de convergência ao vivo
- Linha de KM: traduz o fitness em distância real durante a execução

Painel de rotas por veículo:

- Card por veículo com cor identificadora, número de paradas, carga total e distância da rota
- Lista de cidades com ícones de prioridade (⚠ crítica, ⚡ alta) e demanda

6.3 Análise das Rotas Típicas

Após convergência com os parâmetros padrão (população 100, estagnação 400), o sistema produz tipicamente:

- V1 (vermelho — leve): concentra as cidades do eixo leste da RMSP — Guarulhos, Arujá, Itaquaquecetuba, Suzano, Mogi das Cruzes — aproveitando a continuidade geográfica
- V2 (azul — leve): atende o eixo central e sul — São Paulo, Santo André, São Bernardo, Diadema, Mauá — com alta densidade de pontos críticos
- V3 (verde — pesado/regional): cobre os municípios periféricos de maior distância — Juquitiba, Salesópolis, Guararema, Pirapora — onde a autonomia estendida é necessária

Esta divisão geográfica natural emerge do próprio AG, sem ter sido programada explicitamente — resultado direto da função fitness que penaliza tanto a distância total quanto a fragmentação de rotas.

7. API REST e Interface Web

7.1 Arquitetura da API

A API é implementada com FastAPI, um framework Python moderno que oferece validação automática de parâmetros via Pydantic, documentação interativa Swagger UI gerada automaticamente e suporte nativo a operações assíncronas.

O AG executa em thread separada (threading.Thread), garantindo que a requisição HTTP retorna imediatamente com o job_id. A página web realiza polling a cada 2 segundos em /status/{job_id}, recebendo atualizações incrementais do progresso sem bloquear o servidor.

Método	Endpoint	Descrição
GET	/	Página web principal (HTML)
POST	/otimizar	Inicia o AG em background — retorna job_id
GET	/status/{job_id}	Progresso em tempo real (polling a cada 2s)
GET	/resultado/{job_id}	Resultado completo: rotas + relatório LLM + histórico
POST	/testes/executar	Executa os 58 testes automatizados + gera relatório LLM
GET	/testes/status/{job_id}	Status e resultado do job de testes
GET	/health	Healthcheck — usado pelo Azure para monitoramento
GET	/docs	Swagger UI interativo (documentação automática)

7.2 Gerenciamento de Jobs

Os jobs são armazenados em memória (dicionário JOBS em main.py) com o seguinte ciclo de vida: pending → running → done (ou cancelled). O objeto de job é compartilhado entre a thread do AG e o endpoint de status via referência, eliminando necessidade de banco de dados para esta escala de uso.

7.3 Documentação da API

A documentação interativa Swagger UI está disponível automaticamente em /docs (ex: https://vrp-rmsp.nicerock-a43e5f49.brazilsouth.azurecontainerapps.io/docs), permitindo testar todos os endpoints diretamente no navegador sem necessidade de ferramentas externas como Postman.

8. Testes Automatizados

O projeto conta com 58 testes automatizados organizados em três módulos, cobrindo os componentes críticos do sistema com 100% de aprovação em 0.007 segundos.

Arquivo / Classe	Testes	O que cobre
test_genetic_algorithm.py	22	Crossover OX (permutação válida, sem duplicatas), mutação adjacent swap, ordenação por fitness
test_decoder.py	19	Prioridade crítica > alta > normal, restrição de capacidade, restrição de autonomia, múltiplos veículos
test_fitness.py	17	Fitness multiobjetivo, distância euclidiana (Pitágoras, simetria), refinamento 2-opt
TOTAL	**58**	100% aprovados — tempo de execução: 0.007s

8.1 Estratégia de Testes

test_genetic_algorithm.py (22 testes): Valida os operadores genéticos — o crossover OX é testado quanto à validade da permutação resultante (todos os genes presentes exatamente uma vez), ausência de duplicatas e preservação parcial dos genes dos pais. A mutação é testada para integridade da permutação, imutabilidade do original e comportamento probabilístico correto.

test_decoder.py (19 testes): Testa as restrições de negócio do VRPDecoder — garante que cidades críticas sempre aparecem antes das normais após _sort_by_priority(), que a carga de cada rota nunca excede a capacidade do veículo, que a autonomia é respeitada considerando o retorno ao depósito e que múltiplos veículos são utilizados corretamente.

test_fitness.py (17 testes): Valida a função fitness — retorno de float finito positivo, comportamento correto para casos extremos (cromossomo com um único ponto, frota vazia), cálculo de distância euclidiana (verificado contra o teorema de Pitágoras e propriedade de simetria) e melhoria comprovada do 2-opt sobre a rota original.

9. Arquitetura da Solução em Nuvem (Azure)

9.1 Visão Geral

A solução foi containerizada com Docker e publicada na plataforma Microsoft Azure utilizando Azure Container Apps — um serviço serverless para containers que oferece HTTPS automático, escalonamento automático (0 a N réplicas), integração nativa com Azure Container Registry e monitoramento integrado.

9.2 Recursos Provisionados

Recurso Azure	Nome provisionado	Função
Resource Group	vrp-rmsp-rg	Agrupa todos os recursos do projeto na região Brazil South (São Paulo)
Container Registry (ACR)	vrprmspcr	Armazena a imagem Docker privada da aplicação
Container Apps Environment	vrp-rmsp-env	Ambiente gerenciado com Log Analytics integrado
Container App	vrp-rmsp	Aplicação em execução: 1 CPU, 2 GB RAM, 1–3 réplicas, HTTPS automático

9.3 Fluxo de Deploy

O processo de deploy segue a sequência:

- Build local da imagem Docker via Dockerfile (Python 3.11-slim, dependências pip, cópia dos módulos Python)
- Push da imagem para o Azure Container Registry (ACR) com autenticação via credenciais admin
- Criação do Container App apontando para a imagem no ACR, com configuração de ingress externo na porta 8000
- Injeção da OPENAI_API_KEY como variável de ambiente segura via az containerapp update --set-env-vars

Para atualizações, basta reexecutar os três primeiros passos: docker build → docker push → az containerapp update. O Container Apps realiza o deploy da nova revisão sem downtime (rolling update).

9.4 Containerização

O Dockerfile utiliza a imagem python:3.11-slim como base, minimizando o tamanho final da imagem. O .dockerignore exclui arquivos desnecessários para a API (Pygame, cache de mapas, relatórios gerados, variáveis de ambiente). O comando de inicialização utiliza uvicorn com workers=1 — adequado para o workload de um container App com auto-scaling horizontal.

O Healthcheck integrado ao Dockerfile permite que o Azure monitore a saúde do container e realize reinicializações automáticas em caso de falha, garantindo alta disponibilidade da aplicação.

9.5 URL Pública

A aplicação está disponível em: **https://vrp-rmsp.nicerock-a43e5f49.brazilsouth.azurecontainerapps.io**

O Swagger UI com documentação interativa da API está em /docs. Nenhuma instalação é necessária para utilizar a interface web ou a API — basta acessar a URL no navegador.

10. Entregáveis e Estrutura do Repositório

10.1 Código-Fonte Completo

O repositório contém o código-fonte completo organizado nos seguintes módulos:

- tsp.py: Entry point do modo desktop com loop principal do AG e visualização Pygame
- genetic_algorithm.py: Operadores genéticos (order_crossover, mutate, sort_population)
- core/algorithm.py: Geração da população inicial (aleatória, nearest neighbour, convex-like)
- core/fitness.py: Função fitness multiobjetivo e refinamento 2-opt
- domain/models.py: Entidades do domínio (DeliveryPoint, Vehicle, Route)
- domain/problem.py: VRPProblem — agrega depot, pontos de entrega e frota
- vrp/decoder.py: VRPDecoder — converte cromossomo em rotas respeitando restrições
- benchmark_greater_sp.py: Dataset com 39 municípios da RMSP (coordenadas reais)
- llm_report.py: Integração OpenAI GPT-4o-mini para relatórios de rotas
- generate_test_report.py: Execução de testes + relatório gerencial LLM
- api/main.py: FastAPI com todos os endpoints REST
- api/runner.py: AG headless (sem Pygame) para execução via API
- api/templates/index.html: Interface web completa (Leaflet.js + Chart.js)

10.2 Documentação da API

A documentação da API está disponível em dois formatos:

- Swagger UI interativo: https://vrp-rmsp.nicerock-a43e5f49.brazilsouth.azurecontainerapps.io/docs
- README.md: documentação completa de todos os endpoints com exemplos de uso curl

10.3 Scripts de Deploy

- Dockerfile: Configuração da imagem Docker (Python 3.11-slim, healthcheck integrado)
- deploy_azure.ps1: Script PowerShell para deploy inicial no Azure (primeira vez)
- update_azure.ps1: Script PowerShell para atualização após mudanças no código
- deploy_azure_guia.txt: Guia passo a passo com solução de problemas comuns

11. Conclusão

Este projeto implementou com sucesso uma solução completa para o Problema de Roteamento de Veículos aplicado à distribuição hospitalar na RMSP, atendendo a todos os requisitos do Tech Challenge Fase 2.

O Algoritmo Genético demonstrou ser a abordagem mais adequada para este problema, equilibrando qualidade da solução (dentro de 5-15% do ótimo estimado), tempo de execução aceitável (30-120 segundos) e flexibilidade para incorporar múltiplas restrições operacionais heterogêneas.

A integração com LLMs transforma os resultados numéricos do AG em informações operacionais acionáveis — instruções por motorista, relatórios de eficiência e alertas de prioridade — demonstrando como IA generativa e IA de otimização podem trabalhar em conjunto para entregar valor prático.

O deploy na Azure completa a proposta tornando a solução acessível via web sem instalação local, com HTTPS automático, escalonamento automático e monitoramento integrado — características essenciais para um sistema de apoio à decisão em ambiente hospitalar real.

11.1 Possíveis Evoluções

- Implementação de janelas de tempo (Time Windows VRP) para cada ponto de entrega
- Otimização multi-objetivo com Pareto para equilibrar custo, tempo e prioridade explicitamente
- Persistência de jobs com banco de dados (PostgreSQL/Redis) para histórico de otimizações
- Dashboard administrativo com histórico de rotas e análise de tendências
- Integração com APIs de tráfego em tempo real (Google Maps/HERE) para distâncias reais
- Expansão da suite de testes com testes de integração end-to-end da API